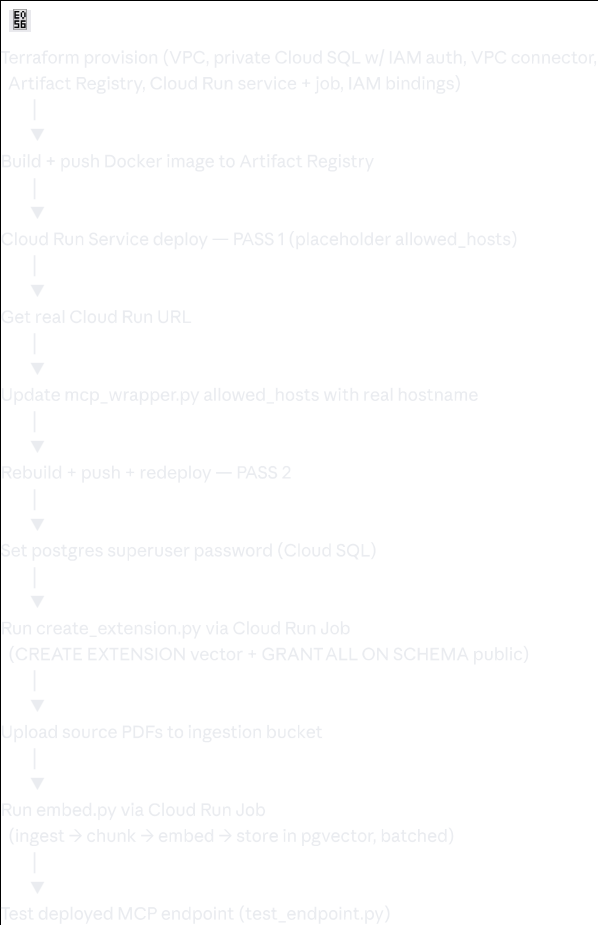


Project 07 (RAG+MCP) — iter3 Dev Rebuild + Dev→Prod Promotion Attempt

Full record: logic flow, steps, and exact commands for (1) rebuilding the RAG+MCP pipeline as `[dev]` in iter3, and (2) the attempted promotion to a `[prod]` environment, including the design flaw discovered and the resulting teardown.

Part 1: Dev Rebuild (iter3) — Flow Logic



Part 1: Dev Rebuild — Steps + Commands

1. Terraform bootstrap

```
EO
SE

bash
gsutil mb -p rag-pipe-mcp-dev -l europe-west4 gs://rag-pipe-mcp-dev-terraform-state
terraform init
terraform plan
terraform apply
```

Recurring transient errors during apply (not config bugs — retry until it succeeds):

DNS/network blips: `[dial tcp: lookup <api>.googleapis.com: no such host]`

If a resource already exists on GCP but not in state (e.g. after a dropped connection mid-apply):

```
EO
SE

bash
gcloud sql instances describe <instance-name> --format="value(state)"
terraform import google_sql_database_instance.rag_pg <PROJECT_ID>/<INSTANCE_NAME>
```

Connector naming rule: `^[a-z][-a-z0-9]{0,23}[a-z0-9]$,` — fix `[name]` in `[cloudrun.tf]` if rejected.

2. Build + push image

```
EO
SE

bash
docker build -t europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest .
docker push europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest
```

(If Docker Desktop fails locally due to no virtualization support, do build/push from Cloud Shell instead.)

3. Cloud Run deploy — pass 1

```
EO
SE

bash
terraform apply # creates Cloud Run service + job once image exists
```

4. Get real URL

```
bash
gcloud run services describe rag-pipe-mcp-dev-mcp-server --region=europe-west4 --format="value(status.url)"
```

5. Update `mcp_wrapper.py`

Add the real hostname (no `https://` prefix — `allowed_hosts` matches the raw `Host` header) to `TransportSecuritySettings.allowed_hosts`.

6. Rebuild + push + redeploy — pass 2

```
bash
docker build -t europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest .
docker push europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest
gcloud run deploy rag-pipe-mcp-dev-mcp-server --image=europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest --region=europe-west4
```

Also update the Cloud Run **Job** to the new image (jobs don't auto-pick-up `:latest` on execute):

```
bash
gcloud run jobs update rag-pipe-mcp-dev-embed-job --image=europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest --region=europe-west4
```

7. Set postgres superuser password

```
bash
gcloud sql users set-password postgres --instance=rag-pipe-mcp-dev-pg --password=<password>
```

8. Run `create_extension.py` via Cloud Run Job

```
bash
gcloud run jobs execute rag-pipe-mcp-dev-embed-job --region=europe-west4 --args=create_extension.py --update-env-vars=POSTGRES_PASSWORD=<password>
```

Bugs hit here:

`role "..."` does not exist on the `GRANT` step → service account email in `create_extension.py` was stale/wrong; fix the SA reference, rebuild/push, `gcloud run jobs update`, re-execute.
Password auth failure → password was never actually set on the instance before running the script; run step 7 first.

9. Upload PDFs to ingestion bucket

```
powershell
gsutil -m cp -r \data* gs://rag-pipe-mcp-dev-ingestion/
```

10. Run `embed.py` via Cloud Run Job

```
bash
gcloud run jobs execute rag-pipe-mcp-dev-embed-job --region=europe-west4
```

Bugs hit here:

`250 instance(s) is allowed per prediction. Actual: 1243` → Vertex AI embedding API caps at 250 texts per call; batch `vectorstore.add_documents()` calls.
`input token count is 59953 but the model supports up to 20000` → count-based batching (250) isn't enough; token count also matters. Reduce `batch_size` to ~25.

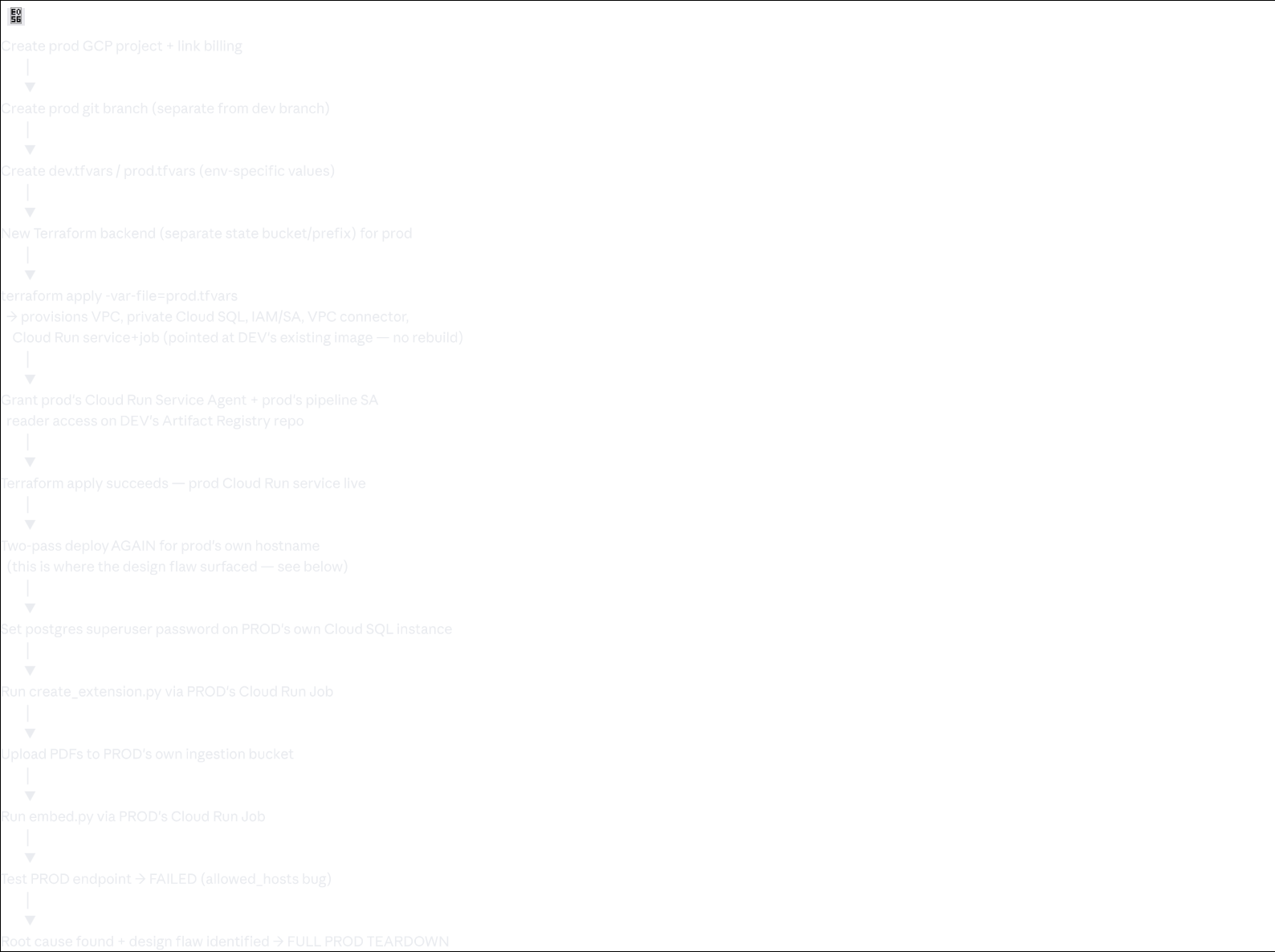
```
python
def embed_and_store(chunks, batch_size=25):
    embeddings = get_embeddings()
    vectorstore = PGVector(embeddings=embeddings, collection_name=COLLECTION_NAME, connection=engine, use_jsonb=True)
    for i in range(0, len(chunks), batch_size):
        vectorstore.add_documents(chunks[i:i + batch_size])
    return vectorstore
```

11. Test the deployed endpoint

```
bash
python3 test_endpoint.py
```

Confirmed working: correct answers with accurate `gs://` source attribution.

Part 2: Dev→Prod Promotion Attempt — Flow Logic



Part 2: Dev→Prod Promotion — Steps + Commands

1. Create prod GCP project + link billing (Console)

Manual step — new project, link billing account (not inherited from dev).

2. Create prod git branch

```
git checkout -b prod
```

3. Create tfvars per environment

```
dev.tfvars:
```

```
project_id      = "rag-pipe-mcp-dev"
region          = "europe-west4"
zone            = "europe-west4-c"
ingestion_bucket_name = "rag-pipe-mcp-dev-ingestion"
db_tier         = "db-f1-micro"
db_name         = "ragdb-dev"
project_number  = "<dev project number>"
```

prod.tfvars: same keys, [-prod] values.

4. New Terraform backend for prod

Update [main.tf] backend block (new bucket/prefix), then:

```
gsutil mb -p rag-pipe-mcp-prod -l europe-west4 gs://rag-pipe-mcp-prod-terraform-state
terraform init -reconfigure
```

5. Point prod's [cloudrun.tf] / [cloudrunjob.tf] at DEV's image

```
hcl
image = "europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest"
```

Delete `artefactregistry.tf` on the prod branch entirely — prod does not provision its own AR repo.

6. Apply Terraform for prod

```
bash
terraform apply -var-file="prod.tfvars"
```

Same transient-error retry pattern as dev (connector DNS blips, Cloud SQL creation drops → `terraform import` if needed).

7. Grant cross-project AR access

Two identities needed reader access — this was a real finding not in the original iter2 docs:

```
bash
# prod's pipeline SA
gcloud artifacts repositories add-iam-policy-binding mcp-server-repo \
--project=rag-pipe-mcp-dev --location=europe-west4 \
--member="serviceAccount:rag-pipe-mcp-prod-pipeline@rag-pipe-mcp-prod.iam.gserviceaccount.com" \
--role="roles/artifactregistry.reader"

# prod's Cloud Run Service Agent (the actual identity that pulls the image at deploy time)
gcloud artifacts repositories add-iam-policy-binding mcp-server-repo \
--project=rag-pipe-mcp-dev --location=europe-west4 \
--member="serviceAccount:service-<prod-project-number>@serverless-robot-prod.iam.gserviceaccount.com" \
--role="roles/artifactregistry.reader"
```

8. Get prod's real Cloud Run URL

```
bash
gcloud run services describe rag-pipe-mcp-prod-mcp-server --region=europe-west4 --project=rag-pipe-mcp-prod --format="value(status.url)"
```

9. THE DESIGN FLAW — what went wrong

`mcp_wrapper.py`'s `allowed_hosts` is hardcoded Python, baked into the one shared Docker image. Adding prod's hostname required editing this file and **rebuilding/repushing to dev's own AR repo** — meaning a "prod-only" need forced a change to the supposedly-frozen, already-tested dev artifact. This defeats the purpose of promotion (same tested artifact, unchanged, across environments).

Compounding bug: prod's hostname entries were added with an `https://` prefix:

```
python
"https://rag-pipe-mcp-prod-mcp-server-528254265815.europe-west4.run.app"

allowed_hosts matches the raw Host header (no scheme) — this entry could never match, causing every prod request to fail with MCPError: Server returned an error response at the
initialize/discover step.
```

Correct fix identified (not implemented before teardown): make `allowed_hosts` environment-agnostic by reading from an env var injected per-environment via Terraform:

```
python
allowed_host = os.getenv("ALLOWED_HOST")
security = TransportSecuritySettings(
    allowed_hosts=[allowed_host, f"{allowed_host}.*"],
    enable_dns_rebinding_protection=True,
```

```
hcl
env {
  name = "ALLOWED_HOST"
  value = "<this-environment's-real-hostname>"
}
```

This still requires a one-time two-pass deploy per environment (hostname unknown until the service exists), but after that, the image itself never needs to change again for a new environment — only Terraform's env var differs.

10. Diagnosing the stale-digest red herring

While chasing the bug, digest comparisons appeared to mismatch:

```
bash
gcloud run revisions describe <revision> --region=europe-west4 --project=rag-pipe-mcp-prod --format="value(spec.containers[0].image)"
gcloud artifacts docker images list europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server --include-tags
```

Resolved as a false alarm — confirmed via:

```
bash
docker buildx imagetools inspect europe-west4-docker.pkg.dev/rag-pipe-mcp-dev/mcp-server-repo/mcp-server:latest
```

The `:latest` tag is a manifest-list digest; Cloud Run resolves and stores the underlying platform-specific (`linux/amd64`) manifest digest — these were the same image, just referenced two different ways. Not an actual staleness bug.

11. Steps 8-11 of the promotion plan (attempted, before teardown)

```
EO
SE

bash
# Set prod's own superuser password
gcloud sql users set-password postgres --instance=rag-pipe-mcp-prod-pg --password=~password~

# create_extension.py on prod's own Cloud SQL
gcloud run jobs execute rag-pipe-mcp-prod-embed-job --region=europe-west4 --project=rag-pipe-mcp-prod --args=create_extension.py --update-env-vars=POSTGRES_PASSWORD=~password~

# Upload PDFs to prod's own bucket (not shown — same gsutil pattern as dev, targeting rag-pipe-mcp-prod-ingestion)

# embed.py on prod's own Cloud SQL
gcloud run jobs execute rag-pipe-mcp-prod-embed-job --region=europe-west4 --project=rag-pipe-mcp-prod

Both create_extension.py and embed.py executions succeeded against prod's own Cloud SQL instance. The MCP endpoint test (test_endpoint.py) pointed at prod's URL) then failed due to the allowed_hosts bug described above.
```

12. Full teardown of prod

Decision made to destroy all of prod's infrastructure and start clean, given the underlying design flaw needed to be fixed before re-attempting promotion.

```
EO
SE

bash
terraform destroy -var-file="prod.tfvars"
```

Hit a real GCP propagation issue during teardown:

```
EO
SE

Error: Unable to remove Service Networking Connection. Producer services (e.g. CloudSQL...) are still using this connection.
```

Diagnosed:

```
EO
SE

bash
gcloud sql instances list --project=rag-pipe-mcp-prod # confirmed empty — instance already gone
gcloud services vpc-peerings list --network=rag-pipe-mcp-prod-vpc --project=rag-pipe-mcp-prod # peering still active
gcloud services vpc-peerings delete --network=rag-pipe-mcp-prod-vpc --project=rag-pipe-mcp-prod # same failure directly via gcloud
```

Root cause: known GCP propagation delay — the Service Networking peering can take significantly longer than the Cloud SQL instance itself to fully release, sometimes hours. Not a config error; no fix available except waiting, or bypassing via full project deletion.

Resolution chosen: delete the entire prod GCP project, since this removes all resources (including ones stuck in propagation limbo) without needing per-resource dependency ordering:

```
EO
SE

bash
gcloud projects delete rag-pipe-mcp-prod
```

Note: project deletion enters a ~30-day pending state before permanent purge; the project can be restored within that window if needed.

Also flagged as needing a check: whether the two bad image digests (with the malformed allowed_hosts) pushed to **dev's** AR repo should be deleted, and whether dev's own currently-running Cloud Run revision was affected by them — dev's own hostname entries in the file were correctly formatted, so dev itself was not confirmed broken, but this was not fully verified before the session's teardown decision.

Key Lessons From This Attempt

True promotion means the image never changes for environment-specific needs. Any config that has to be "known" per-environment (hostnames, URLs, per-env identifiers) belongs in env vars injected by Terraform, not hardcoded in application code — otherwise "promotion" silently becomes "rebuild dev for prod's sake," defeating the purpose of having two environments.

Cross-project image pulls need two separate IAM grants: the app's own service account, AND the Cloud Run Service Agent (service-`<project-number>`@serverless-robot-`prod.iam.gserviceaccount.com`) — the latter is the actual identity performing the image pull at deploy time, easy to miss.

allowed_hosts matches the raw Host header — never include a URL scheme (`https://`) in these entries.

Manifest-list vs. platform-manifest digests can look like a mismatch but aren't — verify with `docker buildx imagetools inspect` before assuming a stale `:latest` deploy.

Service Networking peering deletion can lag Cloud SQL deletion by a long margin — if `terraform destroy` hangs on this specific error and the SQL instance is confirmed gone, it's a genuine propagation wait, not a bug to debug further. Full project deletion is a valid way to force cleanup when time matters more than graceful teardown.